4190.428 001 Introduction to Machine Learning

# Introduction

**Jaesik Park**
CSE & IPAI, SNU
jaesik.park@snu.ac.kr

# Quick notice

- No attendance check during class adjustment period (~10th March)

# Topics that we will cover (tentative)

- Linear Classifiers
- Kernel methods
- Support Vector Machines
- Generalization & Regularization
- Nonlinear Classifiers (Perceptron)
- Gradient Descent
- Regression
- Unsupervised Learning (Clustering, K-means, EM, PCA)
- Sequential Models (Markov decision process)
- Reinforcement Learning (LQR, DDP, LQG, Policy Gradient)
- Non-parametric methods
- Overview of recent approaches

# Machine Learning

TAs took these pictures :)

data

Decides whether a photo is a landscape or a food

Model

- Main Focus of Machine Learning : Making **decisions** or **predictions** based on Data!
- Examples such as Face detection, Speech recognition

# How previous data can predict the future?

- We will make some useful **assumptions** such as :

- All training data are **IID** (identically, independently distributed)
- Queries are drawn from the same distribution as the training data
- Answer comes from a set of possible answers known in advance

# Two main problems of Machine Learning

- **Estimation**:
  - Aggregate noisy data to make accurate predictions
  - Address variability (e.g. same treatment yielding different outcomes)
  - Evaluate how estimates align with future results

- **Generalization**:
  - Predict outcomes for unseen situations or experiments
  - Extend insights beyond the given dataset

# Characterizing Machine Learning Problems & Solutions

- **Problem Characteristics**:
  - Problem class : nature of training data and query types during testing
  - Assumptions : knowledge about data sources or solution forms
  - Evaluation criteria : goals, query evaluation, system performance measures

- **Solution Characteristics:**
  - Model type : the need of Intermediate models, aspects of the data modeled, and model prediction methods
  - Model class : what parametric class of models will be used, selection criteria that selects a particular model from the model class
  - Algorithm : computational process for model fitting and predictions

# Problem Class : Supervised Learning (SL)

- **Supervised Learning**
  - Learning system is given inputs and told which specific outputs should be associated with them (e.g., classification between landscape/food)

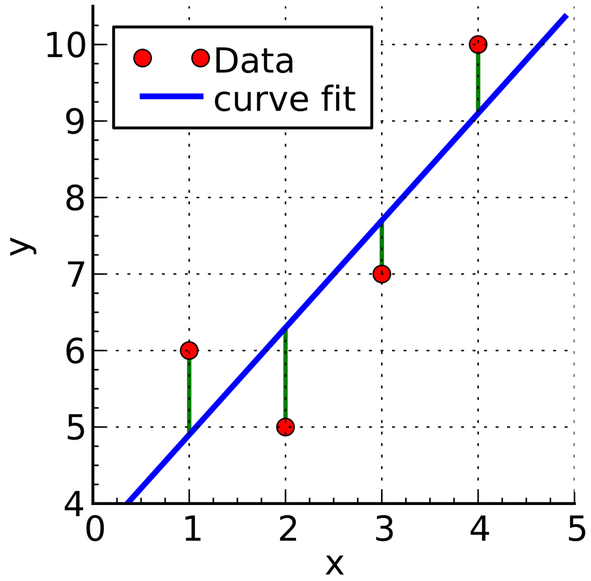
- We divide up supervised learning based on the size of the output set:
  - Classification : Outputs are drawn from a small finite set
  - Regression : Outputs are drawn from a large finite or continuous set

# Problem Class : SL - Classification

- **Classification**
  - Training data $\mathscr{D}_n$: $\{(\mathbf{x}^{(\mathbf{1})}, y^{(1)}), \ldots, (\mathbf{x}^{(\mathbf{n})}, y^{(n)})\}$
    - $\mathbf{x}^{(i)} \in \mathbb{R}^d$ represents an object to be classified (real and/or discrete values)
    - $y^{(i)}$ is called target values: it is an element of a discrete set of values

- Classification problem is **binary** or two-class if $y^{(i)}$ is drawn from a set of two possible values. Otherwise, it is called **multi-class**
  - Goal of classification problem: predict $y^{(n+1)}$ given a new input $\mathbf{x}^{(n+1)}$
  - It is a kind of supervised learning, since $y^{(i)}$ is specified for each of $\mathbf{x}^{(i)}$

# Problem Class : SL - Regression




Linear Regression

- Training data $\mathscr{D}_n$: $\{(\mathbf{x}^{(1)}, \mathbf{y}^{(1)}), \ldots, (\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}$
  - $\mathbf{x}^{(i)} \in \mathbb{R}^d$ represents an object to be classified (real and/or discrete values): Same with classification
  - $\mathbf{y}^{(i)} \in \mathbb{R}^k$: It is different with classification!

# Problem Class : Unsupervised Learning (USL)

- **Unsupervised Learning**
  - It doesn’t involve learning a function from inputs to outputs based on a set of input-output pairs
  - Instead, given a data set, generally it is expected to find some patterns or structure inherent in it

# Problem Class : USL - Density Estimation

- **Density Estimation**
  - Given samples $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)} \in \mathbb{R}^D$, each of it drawn i.i.d from $\Pr(X)$
    - The goal is to predict the probability $\Pr(x^{(n+1)})$ of an element drawn from the same $\Pr(X)$
- It sometimes plays a role as a “subroutine” in the overall learning method for supervised learning

# Problem Class : USL - Clustering

- **Clustering**
  - Given samples $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)} \in \mathbb{R}^D$
  - The goal is to find a **partitioning** (or "**clustering**") of the samples that groups together samples that are similar (depending on the definition of the similarity between samples, and exactly what criterion is to be used)
    - Example : minimize the average distance between elements inside a cluster, maximize the average distance between elements across clusters
  - It is used as a step in density estimation, or to find useful structure in data

- Soft clustering : samples may be assigned 0.9 membership in one cluster, and 0.1 in another

# Problem Class : USL - Dimensionality Reduction

- **Dimensionality Reduction**
  - Given samples $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)} \in \mathbb{R}^D$
  - The goal is to **re-represent** them as points in a d-dimensional space (where $d < D$), and to retain information in the data set (e.g. allow elements of one class to be discriminated from another)

- Note
  - It is a standard technique that is useful for visualizing/understanding high-dimensional data
  - If the goal is to perform regression/classification on the dimensionality reduced data, it is important to consider fundamental variables/ dimensions

# Problem Class : Reinforcement Learning

- **Reinforcement Learning**
  - The Goal is to **learn a mapping** from input values $x$ to output values $y$
    - Without a direct supervision signal to specify which $y$ is best for particular $x$
    - There is no training set specified a $priori$
    - It is framed as an agent interacting with an environment

- Example
  - The agent observes the current state $x^{(0)}$ and selects an action $y^{(0)}$
  - Then it receives a reward $r^{(0)}$ which depends on $x^{(0)}$ (and possibly $y^{(0)}$)
  - The environment makes a transit probabilistically to a new state $x^{(1)}$, with a distribution that depends only on $x^{(0)}$ and $y^{(0)}$
  - Then, the agent again observes the current state $x^{(1)}$, and so on ...

# Problem Class : Reinforcement Learning

- **Reinforcement Learning**
  - The Goal is to **find a policy** $\pi$ that maps $x$ to $y$ (states to actions) such that some long-term sum or average of rewards $\gamma$ is maximized

- This setting is very different from either Supervised/Unsupervised Learning!
- Agent's action choices affect both its reward and ability to observe the environment
- Thus, it requires careful consideration of the long-term effects of actions and all other issues that relative with Supervised Learning

# Problem Class : Sequence Learning

- **Sequence Learning**
  - The Goal is to **learn a mapping** from input sequences $x_0, \cdots, x_n$ to output sequences $y_1, \cdots, y_m$
  - The mapping is typically represented as a state machine :
  - One function $f$ is used to compute the next hidden internal state given the input
  - Another function $g$ is used to compute the output given the current hidden state

- It is 'Supervised' in the sense that we are told what output sequence to generate for which input sequence
- However, Internal functions have to be learned by some method (Different With Direct Supervision) as we don't know the hidden state sequence

# Problem Class : Other Settings

- **Semi-Supervised Learning**:
  - Supervised-learning Training Set + Additional set with $x^{(i)}$ with unknown $y^{(i)}$

- **Active Learning**:
  - Used when it is expensive to acquire a label $y^{(i)}$ (e.g. X-ray Image)
  - Learning Algorithm can sequentially ask for particular inputs $x^{(i)}$ to be labeled

- **Transfer Learning (Meta-Learning)**:
  - There are multiple tasks with data drawn from different (but related) distributions
  - The goal is to use prior task experience to accelerate learning of a new task, with less effort

# Assumptions

- These **assumptions** are often made about the data source or the solution :
  - The data follows i.i.d assumption (independent and identically distributed)
  - The data are generated by a Markov chain
  - The process that generates the data might be adversarial
  - The ‘true’ model that generates the data can be perfectly described by one of some particular set of hypotheses

- The effect of an assumption is often to reduce the size/expressiveness of the space of possible hypotheses —> Reduce the amount of data required

# Evaluation Criteria

- What makes an output or the answer to a query, '**good**'?

- We specify evaluation criteria at two levels:
  - How an **individual prediction** is scored, and how the **overall behavior** of the prediction or estimation system is scored
  - The quality of predictions from a learned model: often expressed by Loss Function $L(g, a)$, how much penalty for making a guess $g$ when the answer is actually $a$

# Evaluation criteria : Loss Function Examples

- 0-1 Loss :
  - applied to predictions drawn from finite domains

$$L(g, a) = \begin{cases} 0, & \text{if } g = a \\ 1, & \text{otherwise} \end{cases}$$

- Squared Loss :

$$L(g, a) = (g - a)^2$$

- Linear Loss :

$$L(g, a) = |g - a|$$

# Evaluation criteria : Loss Function Examples

- Asymmetric Loss :
  - when it is much worse to predict “no” when the answer is actually “yes”.
  - e.g. Does this food causes allergy in some people?

$$L(g, a) = \begin{cases} 1, & \text{if } g = 1 \text{ and } a = 0 \\ 10, & \text{if } g = 0 \text{ and } a = 1 \\ 0, & \text{otherwise} \end{cases}$$

# Evaluation criteria

- Any given prediction rule will be evaluated based on multiple predictions and the loss of each one. During evaluation, we might consider:
- **Minimizing expected loss** over all the predictions (Minimizing risk) :
  Most of the time, we will concentrate on this criterion!

- Minimizing maximum loss
- Minimizing or bounding regret
- Characterizing asymptotic behavior
- Finding algorithms that are probably approximately correct

# Model type

- No Model: in regression/classification, we can just averaging answers to recent queries as in the nearest neighbor method.

- Prediction rule:
  - 1. “Fit” a model to the training data
  - 2. Use the model directly to make predictions
- The model will be some hypothesis or prediction rule $y = h(x; \theta)$
  - $\theta$ is a vector of one or more parameter values
- Given a new $x^{(n+1)}$, we would then make the prediction $h(x^{(n+1)}; \theta)$

# Model type

- The Fitting Process is often articulated as an **optimization problem**:
  - Find a value of $\theta$ that minimizes some criterion
- Optimal strategy : When we know the actual distribution of the data, Pr$(X, Y)$, then minimize the expected loss (test error)
  - If we don't have it, we minimize the training error : find $h$ that minimizes the average loss on our training data set
  - That is, find $\theta$ that minimizes $\epsilon_n(\theta) = \frac{1}{n}\sum_{i=1}^{n} L(h(\mathbf{x^{(i)}}; \theta), y^{(i)})$

    where $L(g, a)$ is a loss function : But we still have to consider overfitting!

# Model Class / Parameter Fitting

- **Model class** $\mathscr{M}$: Set of possible models, typically parameterized by a vector of parameters $\Theta$
  - Ex) Regression problem : $h(\mathbf{x}; \theta, \theta_{\mathbf{0}}) = \theta^\top \mathbf{x} + \theta_0$ , $\mathbf{\Theta} = (\theta, \theta_0)$
- We'll spend a lot of time exploring these model classes!

- **Parametric models**: Models with Fixed, Finite number of parameters
  - (Cf.) Non-parametric models
- Model-Selection Problem : to pick a model class $\mathscr{M}$
- Model Fitting : Pick a particular model in that class, specified by parameters $\theta$

# Algorithm

- What sequence of computational instructions should we run in order to find a good model?
  - Ex.) determining the parameter $\theta$ that minimizes $\epsilon_n(\theta)$ by least-squares minimization
- Some algorithms are not easily seen as trying to optimize a particular criterion
  - Such as Perceptron Algorithm!

# QnA

- Any questions?
- Note: this lecture notes are largely adopted from
  - MIT 6.036(2019 Fall), Stanford CS229(2023-2024 Summer)